# Annexe A18 — OBJ, MTL et ResourceManager

LibraryV3 · Phase G, étapes 3.3e, 3.4a, 3.4b, 3.4c · qui porte quoi, LOD contre mipmap, les clés des caches

**Résultat en une ligne.** Chaque information d'apparence vit à **un seul niveau** (géométrie, matériau ou instance) ; chaque asset a **un** MTL partagé par toute sa chaîne de LOD ; et les trois caches du ResourceManager (meshes, matériaux, bibliothèques MTL) ont enfin une clé **complète**. Un objet demandé est désormais exactement l'objet obtenu, et un MTL n'est lu qu'une fois : 18 → 9 lignes `[MTL]` au chargement.

## 1. Trois niveaux, trois responsabilités




Figure 1 — Chaque information vit à un seul niveau. Une texture est un paramètre de matériau, jamais une propriété d'instance.

| Niveau | Fichier | Ce qu'il décrit | Équivalent Unity |
|---|---|---|---|
| Géométrie | `.obj` | sommets, faces, UV, et **quel** matériau chaque groupe de faces utilise (`usemtl`) | Mesh |
| Apparence | `.mtl` | `Kd`, `Ks`, `Ns`, `map_Kd`, `map_Bump`, `map_Ke`… | Material |
| Instance | scène | **cet** objet, **ce** mesh, éventuellement un **autre** matériau | MeshRenderer |

La scène v1 portait une clé `texture` sur 641 meshes, jamais lue. C'était un matériau réduit à un seul champ : le générateur connaît pour la Terre une albedo, une normal map, une height map, un masque spéculaire, une emissive et des nuages. **D3** : la clé est supprimée ; l'information n'est pas perdue, elle est dans `gen_solar_system.py`.

### 1.1 Pourquoi le MTL du mesh ne suffit pas pour les planètes

Tous les astres partagent trois géométries (`sphere_hi`, `sphere_mid`, `sphere_lo`) : c'est la base du cache et des chaînes de LOD. Un `.obj` désigne **un** MTL : la Terre, Mars et Mercure, toutes en `sphere_mid`, ont donc forcément le même matériau.

> × Dupliquer la géométrie pour changer d'apparence : `earth.obj` = copie de `sphere_mid.obj` + `mtllib earth.mtl`, et ainsi de suite. 33 copies de la même sphère, 33 entrées de cache, 33 chaînes de LOD à écrire, et chaque correction à faire 33 fois. On paierait une donnée d'apparence en dupliquant de la géométrie.

La solution est celle de Unity : une **surcharge de matériau sur l'instance**, par nom, `"Mesh": { "model": "sphere_mid.lod.json", "material": "M_Earth" }`. Sans clé `material`, c'est le `usemtl` du `.obj` qui s'applique. C'est le programme de **L08** ; on ne l'anticipe pas.

## 2. D9 : un MTL par asset, et le mipmap pour la résolution

**Le principe (Onky).** Chaque asset a son propre MTL : c'est la convention OBJ (`rock_a.obj` va avec `rock_a.mtl`), et c'est ce qui rend un asset **autonome**, copiable sans casser un fichier partagé. Si deux matériaux utilisent la même image, ce n'est pas aux fichiers de partager quoi que ce soit : c'est au **ResourceManager** de charger l'image une seule fois.

**La nuance : un MTL par *asset*, pas par *fichier* OBJ.** Une chaîne de LOD est **un** asset : `rock_a_L1.obj`, `rock_a_L2.obj` et `rock_a_L3.obj` référencent `rock_a.mtl`.

> × Quatre MTL identiques pour les quatre niveaux. Un jour, on modifie le `Kd` de `rock_a.mtl` et on oublie `rock_a_L2.mtl` : en s'éloignant, le rocher **change de couleur** au passage en L2. Un popping de matériau se voit bien plus qu'un popping de géométrie.

**L'invariant : un changement de niveau de LOD ne change que la géométrie, jamais l'apparence.** Un seul MTL par chaîne le garantit par construction. Les sphères forment une seule famille (`sphere_mid` est à la fois L1 de `sphere_hi` et L0 de `sphere_mid`), donc un seul `sphere.mtl`.

| Fichier | Référencé par | Matériau |
|---|---|---|
| `rock_a.mtl`, `rock_b.mtl`, `rock_c.mtl` | `rock_x.obj` et `rock_x_L1..L3.obj` | `M_Rock_A`, `M_Rock_B`, `M_Rock_C`, avec `map_Kd Textures/Divers/rock_albedo_1k.jpg` |
| `sphere.mtl` | toutes les sphères (`hi`, `mid`, `lo`, `12x6`, `8x5`, `6x4`) | `M_Sphere` |
| `ship_scout.mtl` | `ship_scout.obj` | 6 maps, chemins `Textures/Ship/…` |

### 2.1 LOD géométrique contre mipmap

Onky a soulevé la bonne objection : avec un seul MTL par chaîne, le niveau L3 d'une sphère (quelques pixels à l'écran) lirait la même texture 2048 × 1024 que L0. Le problème est réel, mais la réponse n'est pas un MTL par niveau : c'est **le mipmap**.

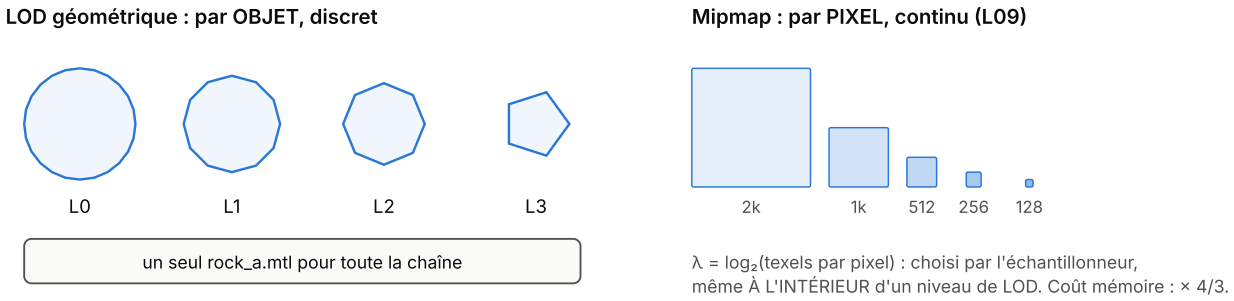



Figure 2 — Deux LOD indépendants. Le LOD géométrique réduit les faces d'un objet entier ; le mipmap réduit la résolution de texture pixel par pixel. Dupliquer les MTL par niveau referait le mipmap à la main, en pire.

| | LOD géométrique | Mipmap (L09) |
|---|---|---|
| Ce qu'il réduit | le nombre de faces | la résolution de la texture |
| Critère | erreur géométrique projetée, en pixels (ε, τ) | texels par pixel (ρ) |
| Granularité | **par objet**, une fois par frame et par vue | **par pixel** |
| Transition | discrète : L1 puis L2 | continue (filtrage trilinéaire) |
| Qui décide | `SelectLodLevel` | l'échantillonneur, à partir des dérivées des UV |

Exemple chiffré : une planète de 10 pixels de large, albedo 2048 × 1024. La moitié visible montre environ 1 024 texels horizontalement sur 10 pixels : ρ ≈ 100, λ = $\log_2$ ρ ≈ 6,6, l'échantillonneur lit le **mip 7** (16 × 8

texels). La texture haute définition sur le petit LOD n'est donc **pas** lue en haute définition, et le choix se fait même **à l'intérieur** de L0, au bord d'une planète vue en biais. Sans mipmap, ce pixel prendrait 1 texel sur les 100 qu'il couvre, différent à chaque frame : la planète **scintille**. C'est ce scintillement, l'aliasing de minification, qui est le vrai danger, pas la mémoire.

> × Une map par niveau dans des MTL séparés (`rock_2k.jpg` en L0, `rock_1k.jpg` en L1…). La netteté saute d'un coup au changement de niveau, en même temps que la géométrie ; le scintillement reste à l'intérieur de chaque niveau ; et on refait à la main, en moins bien, une chaîne de mips qui ne coûte que +33 % de mémoire (1 + ¼ + 1/16 + … = 4/3), calculée une fois au chargement. L'économie de mémoire relève du **streaming de mips** : c'est bien le rôle du ResourceManager, mais au niveau de la texture.

Les cas où un matériau par niveau est légitime existent (Unreal le permet) et ne concernent jamais la résolution : un **shader allégé** au loin (sans normal map), un **imposteur** (un quad face caméra), ou des **UV déplacées** par un futur simplificateur QEM. Si l'un arrive, la surcharge sera déclarée dans le `.lod.json`, contrôlée par `WarnUnread` :

```
{ "mesh": "rock_a_L3.obj", "epsilon": 0.12, "material": "M_Rock_A_Far" }
```

## 3. Les chargeurs ne se taisent plus (pas 3.3e)

`MTLLoader` et `OBJLoader` perdaient des informations sans un mot. Avec **le code seul**, la scène active a révélé 42 soucis :

| Souci révélé | Nombre | Cause |
|---|---|---|
| `[MTL] fichier introuvable : …planet.mtl` | 18 | le fichier n'avait jamais existé dans le dépôt |
| `[OBJ] usemtl 'M_Rock'` / `'M_Planet'` introuvable | 18 | conséquence du précédent |
| `[MTL] texture introuvable : … Meshes/assets/Textures/ship_scout_*.png` | 6 | un chemin MTL est relatif **au dossier du MTL** ; les textures étaient dans `Meshes/Textures/Ship/` |

Ce que fait le chargeur depuis : MTL introuvable → souci ; `usemtl` inconnu → souci, **un par nom** (et pas pour un nom vide : les groupes `g` créent aussi des entrées) ; directive avant `newmtl` ou directive inconnue → souci avec `fichier:ligne` ; `map_Bump` et `map_Ke` pris en charge ; chaque texture référencée est **vérifiée** dès aujourd'hui (absente → souci, présente → info « non chargée avant L09 »).

> **À retenir.** On vérifie une **référence** dès qu'elle existe, même si la ressource ne sera chargée que plus tard. Un chemin faux trouvé en L09 coûte une heure de recherche dans le chargeur d'images ; trouvé aujourd'hui, il coûte une ligne.

Les générateurs ont suivi : une seule définition des matériaux dans `gen_meshes.py` (`SPHERE_MTL`, `rock_mtl()`), importée par `gen_rock_lod.py` et `gen_sphere_lod.py`. `gen_ship.py` produisait des UV qui se recouvraient (124 triangles : bouchons trop larges, ailes sur la dérive) ; corrigé sur 5 valeurs, il reproduit le `ship_scout.obj` du dépôt **octet par octet**.

## 4. Les clés des caches (pas 3.4)

Les trois défauts du ResourceManager ont une seule racine, qui vaut pour tout cache :

**La clé d'un cache doit contenir tout ce qui change le résultat.** Si deux demandes différentes peuvent donner la même clé, le cache rend le résultat de la première à la seconde, sans un mot.

### 4.1 Meshes et chaînes : chemin + options (3.4a)

`LoadMeshChecked(path, opt)` rangeait le mesh sous `CanonicalKey(path)`. Or `scale`, `flipUVsVertically`, `flipWindingOrder` et les options de normales changent les données. Demander `rock_a.obj` avec `scale = 2` rendait le mesh à l'échelle 1. `LoadLodChainChecked` avait le même défaut.

```
// OBJLoadOptions.h
[[nodiscard]] std::string CacheKey() const
{   // les 4 booleens + les 32 bits EXACTS du float
    return std::format("|n{}s{}w{}u{}x{:08X}", int(generateNormalsIfMissing),
int(generateSmoothNormals),
                       int(flipWindingOrder), int(flipUVsVertically),
std::bit_cast<std::uint32_t>(scale));
}
static_assert(sizeof(OBJLoadOptions) == 8, "OBJLoadOptions a change : mettre CacheKey() a
jour");
```

Avec les options par défaut, la clé devient `C:/…/rock_a.obj|n1s0w0u1x3F800000`, où `3F800000` est l'écriture binaire exacte de 1.0f. `FindMesh` et `IsMeshLoaded` prennent désormais les options, **sans valeur par défaut**.

> × `std::to_string(scale)` dans la clé : 6 décimales, donc `1.0f` et `1.0000001f` donnent la même clé. On met les bits exacts (`std::bit_cast`).

> × Garder `FindMesh(path, opt = {})`. `TestLodChain` charge avec `flipUVsVertically = false` puis cherche avec les options par défaut : la recherche aurait rendu `Invalid`, en silence. Sans défaut, le compilateur liste chaque appel à corriger.

> × Ajouter un champ à `OBJLoadOptions` sans mettre la clé à jour : le bug revient. Le `static_assert` sur `sizeof` fait échouer la compilation (4 booléens + 1 float = 8 octets ; un champ de plus change la taille).

### 4.2 Matériaux : MTL + nom (3.4b)

`RegisterMaterial` rangeait le matériau sous son **nom seul**. Blender appelle par défaut tous ses matériaux `Material` : le casque rouge et le bouclier vert, chacun avec son MTL, recevaient tous deux le rouge. D9 contournait le problème par des noms uniques (`M_Rock_A`…), mais une convention sur les données ne protège pas du premier OBJ importé tel quel.

La clé devient `"<MTL canonique>#<nom>"`, fabriquée en **un seul endroit** (`ResourceManager::MaterialKey`), utilisé par `RegisterMaterial` et par `FindMaterial(mtlPath, name)`. Le nom affiché du matériau reste `Material` : l'identité appartient au cache, pas à l'objet.

### 4.3 Bibliothèques MTL : lues une fois (3.4c)

`OBJLoader` relisait chaque MTL pour chaque OBJ : 4 fois par chaîne de LOD. Avant d'écrire le cache, il a fallu supprimer une dépendance cachée : `MTLLoader::Load` résolvait les textures depuis le dossier **de l'OBJ** (`baseDir`). Cacher le MTL sur son seul chemin aurait laissé le premier OBJ fixer la résolution des textures pour tous les suivants. Deux solutions : mettre `baseDir` dans la clé, ou supprimer la

dépendance. On a supprimé : les textures sont désormais résolues depuis le dossier **du MTL** (ce que font Blender et la plupart des chargeurs), et la clé « chemin canonique du MTL » est complète.

```
MaterialMap ResourceManager::LoadMaterialLibrary(const std::string& mtlPath)
{
    const std::string key = CanonicalKey(mtlPath);
    if (auto it = m_pathToMtl.find(key); it != m_pathToMtl.end()) return it->second;
    MaterialMap result = MTLLoader::Load(mtlPath, *this);
    if (fs::exists(mtlPath)) m_pathToMtl.emplace(key, result);   // on ne cache QUE les succes
    return result;
}
```

Conséquence utile : un nom déjà vu **dans la même lecture** ne peut plus venir que d'un `newmtl` écrit deux fois ; c'est maintenant un souci, et la première définition est gardée.

> × Un cache `static` dans `MTLLoader` : il survivrait à `UnloadAll` et serait partagé entre deux ResourceManager. Le propriétaire des ressources possède aussi le cache.

> × Mettre en cache un MTL introuvable : le premier OBJ serait averti, les suivants non. On ne met en cache que les succès, comme `LoadMeshChecked` et `LoadLodChainChecked`.

## 5. Les fichiers eux-mêmes : versionnés et nommés une seule fois

- Le `.gitignore` du modèle Visual Studio contient `*.obj` (prévu pour les fichiers objets du compilateur) : il excluait les **maillages**. Exception ajoutée : `!assets/Meshes/*.obj`.
- `Cube.obj` / `cube.obj` / `mtllib Cube.mtl` : Windows ignore la casse, Git non. Tout est aligné en minuscules (`git mv`, pas l'Explorateur, qui ne voit pas un changement de casse seul).
- Les deux MTL issus de 3ds Max sont nettoyés (`Tf` non lu, texture en chemin absolu) ; leurs noms `newmtl` sont gardés, puisque les OBJ y font référence.

## 6. Aide-mémoire

| Question | Réponse |
|---|---|
| Où mettre une texture | dans le MTL de l'asset ; jamais dans la scène |
| Un objet plus petit à l'écran | le LOD réduit les faces ; le mipmap (L09) réduit les texels |
| Un MTL pour chaque niveau de LOD ? | non : un MTL par asset ; une surcharge exceptionnelle se déclare dans le `.lod.json` |
| Chemins dans un MTL | relatifs au dossier **du MTL** |
| Clé du cache des meshes | chemin canonique + `OBJLoadOptions::CacheKey()` |
| Clé d'un matériau | `<MTL canonique>#<nom>` |
| Clé d'une bibliothèque MTL | chemin canonique du MTL |
| Pointeur `GetLodChain` | temporaire : on garde le handle (A16 § 6) |